

BEYOND^X LLC

TECHNICAL PAPER 004 | PUBLICATION EDITION



Mainframe Data Architecture - IMS

**Hierarchical Discovery, Recursive Data Accountability, COBOL DML
Transformation, and AI-Assisted Modernization**

*A practical engineering methodology for understanding and transforming IMS databases and
their associated application ecosystems to relational and cloud platforms.*

Prepared by Beyond^X LLC

Technology with Purpose. Opportunity with Impact.

Publication Edition - September 2026

Executive Summary

IBM IMS modernization is not a database unload-and-load exercise. The source architecture is distributed across DBDGEN definitions, PSBGEN application views, PCBs and SENSEGs, PCB masks, Segment I/O Areas, COBOL or PL/I copybooks, Segment Search Arguments (SSAs), DL/I calls, JCL, change-control artifacts, runtime behavior, and the data itself. Long-lived systems may also contain definitions that remain physically present after their business use and external documentation have disappeared.

This publication presents a recursive, auditable modernization method. Each DBD can transform into a set of relational tables; each child segment may itself have descendants that transform into additional tables. Every source segment, row population, data element, parent/child relationship, application access path, and disposition must be accounted for. 100% accounted does not mean 100% migrated: data may be migrated, transformed, consolidated, archived, retained, or explicitly excluded, but no population should disappear without evidence and approval.

AI tools can accelerate discovery, correlation, code understanding, candidate mapping, DML transformation, and test generation. They do not replace source evidence, reconciliation, or engineering judgment.

1. IMS Hierarchy Is Recursive, Not Flat

IMS organizes data as a hierarchy of segments. A root can have children; those children can have their own children, continuing through multiple generations. Modernization therefore requires recursive hierarchy discovery and recursive target accountability.

```
DBD
|
+-- ROOT-SEG      -> ROOT_TABLE
|
| +-- CHILD-A-SEG  -> CHILD_A_TABLE
| |
| | +-- CHILD-A-1-SEG -> CHILD_A_1_TABLE
| | | +-- A-1-1-SEG -> CHILD_A_1_1_TABLE
| | |
| | +-- CHILD-A-2-SEG -> CHILD_A_2_TABLE
| |
| +-- CHILD-B-SEG   -> CHILD_B_TABLE
|   +-- CHILD-B-1-SEG -> CHILD_B_1_TABLE
```

The target need not mechanically reproduce IMS navigation. It must preserve the business relationships and ancestry required by the applications and data. Immediate-parent keys normally become explicit relational relationships, while higher ancestry may be retained where business semantics, performance, auditability, or access patterns require it.

2. DBDGEN, PSBGEN, PCB, and SENSEG Discovery

Discovery should begin by inventorying the IMS control-block source and reconciling it with application and production evidence. DBDGEN establishes physical database structure; PSBGEN establishes application views; PCBs identify database access contexts; SENSEGs define segments visible through those program views. The discovery question is not only what is defined, but what is exposed, referenced, populated, and still operationally meaningful.

Evidence	Primary discovery value
DBDGEN	DBDs, segment hierarchy, fields, keys/access definitions and physical relationships.
PSBGEN	Application views of IMS resources and database access definitions.
PCB / SENSEG	Program database context and the segments exposed to an application.
PCB Mask	Program-side communication/status structure used with IMS calls.
Segment I/O Area	How returned segment bytes are interpreted by the program.
SSA	Actual segment qualification and navigational search intent.
COBOL / PL/I	Business rules, DL/I calls, status handling and data interpretation.
JCL / change control / runtime	Execution lineage, batch dependencies, deployed behavior and gaps in maintained source.
Actual IMS data	What populations and relationships physically exist.

Every DBDGEN-defined segment and every PSBGEN/PCB/SENSEG application view should be reconciled against program usage and actual data. Metadata existence is not proof of active use, and missing documentation is not proof of non-use.

3. Segment I/O Areas and the Copybook Parsing Challenge

Segment I/O copybooks can be among the most important surviving specifications of how an application interprets IMS data. Parsing becomes particularly difficult when nested OCCURS, REDEFINES, discriminator fields, group items, packed numeric fields, and historical layouts are combined.

```
01 EMPLOYEE-INFO.  
  03 EMPLOYEE OCCURS 50 TIMES.  
    05 NAME PIC X(30).  
    05 AGE PIC 9(3).  
03 NEW-EMPLOYEE REDEFINES EMPLOYEE.  
  04 G2 OCCURS 50 TIMES.  
    05 FNAME PIC X(10).
```

```
05 LNAME PIC X(20).
05 AGE PIC 9(3).
```

OCCURS defines repetition. REDEFINES provides alternate interpretations of the same storage. A discriminator such as CLAIM-TYPE may determine which interpretation is valid and which target table receives the transformed data. A conversion engine must therefore understand record context, not simply expand every layout as though it represented independent physical data.

```
CLAIM-TYPE = 'P' -> interpret professional layout -> PROFESSIONAL_CLAIM
CLAIM-TYPE = 'I' -> interpret institutional layout -> INSTITUTIONAL_CLAIM
```

Where external documentation has been lost, these I/O definitions may become forensic evidence. They must be correlated with program logic, SSAs, actual data values, and runtime usage before a target mapping is accepted.

4. SSAs and IMS DML Reveal Application Semantics

Segment Search Arguments show how programs qualify and navigate IMS segments. DL/I calls reveal the operation being performed. Together with the PCB and I/O area, they describe behavior that cannot be recovered reliably from a DBD alone.

```
CALL 'CBLTDLI' USING W-GU
      DB-PCB-MASK
      CUSTOMER-SEG
      CUST-SSA.

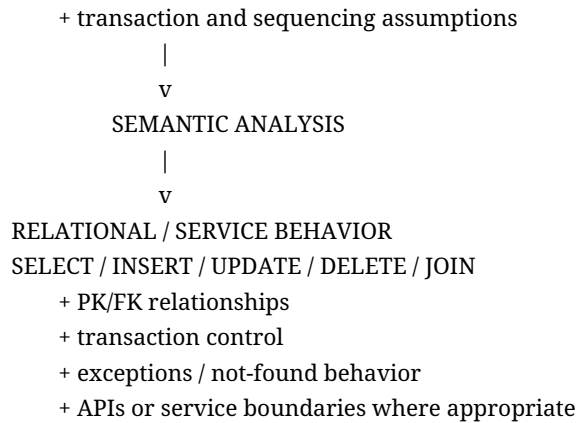
IF PCB-STATUS-CODE = SPACES
  DISPLAY 'Customer Name: ' CUST-NAME
ELSE
  DISPLAY 'Error reading IMS Database. Status: ' PCB-STATUS-CODE
END-IF.
```

This example contains several semantics that a relational transformation must preserve: the Get Unique intent, database/PCB context, the CUSTOMER segment I/O area, SSA qualification, returned data interpretation, and IMS status handling. The correct relational implementation is therefore not merely a syntactic replacement with SELECT.

5. Transforming IMS DML to Relational DML

IMS access is navigational; relational access is declarative. Transforming application DML is frequently harder than moving the data because the new implementation must reproduce business behavior without carrying forward unnecessary physical navigation assumptions.

```
IMS APPLICATION BEHAVIOR
GU / GN / GNP / ISRT / REPL / DLET
+ PCB context
+ SSA qualification
+ segment I/O interpretation
+ status / error handling
```



A GU with an SSA may become a parameterized SELECT, join, stored procedure, repository call, or service request depending on the target architecture. Repeated GN/GNP patterns may imply cursor-like iteration, ordered result processing, or set-based SQL. ISRT, REPL, and DLET require equivalent integrity, transaction, and error semantics. The goal is behavioral equivalence, not statement-by-statement translation.

6. Recursive Source-to-Target Accountability

Each DBD should have a formal source-to-target control model. Because every child can have descendants, accountability must continue recursively from root to every leaf. For every segment path, the team must document the target table or other disposition, the relationship to its immediate parent, and the treatment of every data element.

Control level	Required accountability
DBD	Every in-scope DBD has an approved target or disposition.
Segment	Every segment at every hierarchy level is inventoried and dispositioned.
Row population	Counts/populations are computed, transformed as required, and reconciled.
Data element	Every source field is mapped, transformed, archived, retained, or explicitly excluded.
Relationship	Immediate-parent and required ancestor relationships are validated.
OCCURS / REDEFINES	Expansion and alternate-layout logic is explained and reconciled.
Application view	PSB/PCB/SENSEG exposure is reconciled with physical DBD structure.
Application behavior	SSAs, DL/I calls, status handling, and business logic have a target disposition.

A simple source-row-count equals target-row-count rule is insufficient. OCCURS can expand one source structure into multiple rows; subtype logic can route records to different tables; multiple legacy structures can consolidate into a canonical entity. The reconciliation equation is therefore based on documented transformation rules and populations, not naive one-to-one counts.

SOURCE POPULATION
= MIGRATED
+ TRANSFORMED / EXPANDED
+ CONSOLIDATED
+ ARCHIVED
+ RETAINED
+ EXPLICITLY EXCLUDED

Every difference must have evidence, a rule, and an approved disposition.

7. Data-Element Audit Matrix

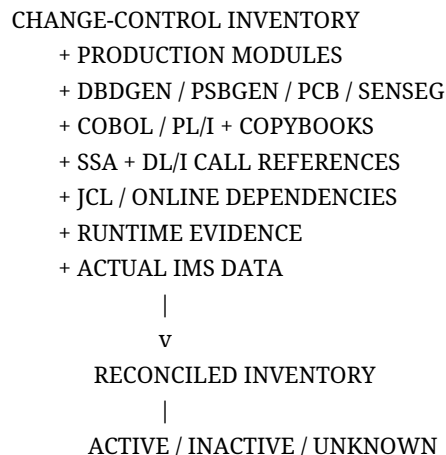
Row reconciliation alone is not enough. Column-level or data-element-level accountability is required to prove that business information has not silently disappeared during flattening or application redesign. A practical control matrix should include the following fields.

Source / Control	Target / Validation
DBD / Segment / hierarchy path	Target table or disposition
Source field / data element	Target column(s)
PIC / source data type / length	Target type / precision / length
OCCURS / occurrence rule	Expansion/cardinality rule
REDEFINES / discriminator	Active-layout decision rule
Parent and immediate parent	PK/FK or relationship mapping
Transformation rule	Conversion/default/derivation rule
Validation rule	Counts, nulls, domain, totals, hashes/samples, relationship checks
Status	Mapped / transformed / archived / excluded / unknown

The objective is a provable disposition for every source data element and every relationship, with exceptions tracked to closure.

8. Discovery When Change Control and Documentation Are Incomplete

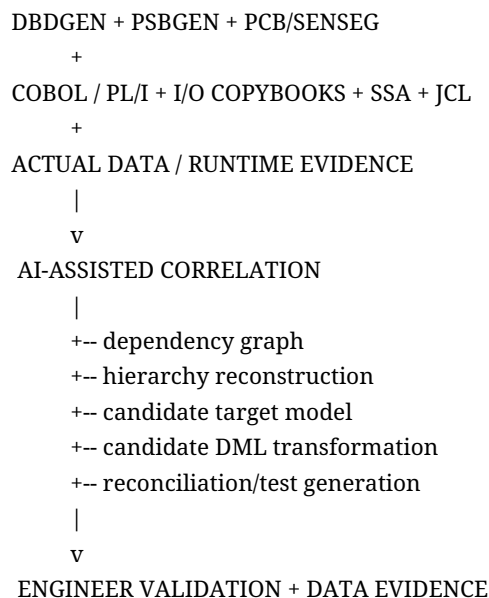
Long-lived IMS environments may contain production modules, copybooks, DBD/PSB source, JCL, or historical layouts whose lineage is incomplete. Some metadata may no longer be used; other artifacts may still be operational even though their external documentation has disappeared. Discovery must therefore be evidence-based.



UNKNOWN is a legitimate control status. It should trigger investigation rather than an assumption that the artifact is obsolete. The modernization team should not discard metadata or data solely because current staff do not recognize it.

9. AI-Assisted IMS and COBOL Modernization

AI can accelerate correlation across thousands of legacy artifacts. It can identify DBD-to-PSB-to-program dependencies, parse COBOL copybooks, detect OCCURS/REDEFINES patterns, locate CBLTDLI calls and SSAs, propose source-to-target mappings, suggest candidate SQL or service logic, and generate test cases. This is particularly valuable where documentation is fragmented.



AI output should be treated as a candidate interpretation until it is validated against source definitions, actual populations, application behavior, and reconciliation controls. Tools accelerate modernization, but they do not replace engineering judgment.

10. Testing: Proving Behavioral and Data Equivalence

Testing must validate both data equivalence and application behavior. A converted query that returns data is not sufficient evidence that the IMS semantics were preserved. For representative and boundary conditions, the same business input should be executed against the legacy behavior and the modern implementation and compared.

Validation area	Examples
Data population	Segment counts, transformed table counts, subtype populations, OCCURS expansion totals.
Data elements	Values, null/default behavior, precision, dates, encoding, discriminator interpretation.
Relationships	Immediate-parent keys, ancestor context, orphan detection, cardinality.
Read behavior	GU/GN/GNP result equivalence, qualification, duplicates, ordering where meaningful.
Write behavior	ISRT/REPL/DLET effects, integrity, transaction outcome, rollback behavior.
Status/error behavior	Not found, invalid qualification, database/application errors and retries.
Business controls	Financial/control totals, business measures, audit populations and exception reports.

Automated comparison is valuable, but high-risk transformations also require manual review of mapping logic, exception populations, and representative records. Testing should prove that hierarchical interpretation and navigational behavior have been faithfully reconstructed, not merely that the new code compiles or SQL executes.

11. The 100% Accountability Principle

The objective is not to migrate everything blindly. It is to account for everything before deciding what must be migrated, transformed, archived, retained, consolidated, or retired. The scope includes metadata, data, relationships, and application access behavior.

100% SOURCE SCOPE

- |
- +-- every DBD / DBDGEN definition
- +-- every segment and descendant path
- +-- every PSB / PCB / SENSEG view
- +-- every relevant data element
- +-- every populated source population
- +-- every parent/child relationship
- +-- every relevant I/O layout / OCCURS / REDEFINES rule
- +-- every relevant SSA / DL/I access pattern
- +-- every approved exception or retirement decision

|
v
100% ACCOUNTED AND AUDITABLE

Modernization is complete only when the agreed source scope has a documented and validated disposition and the legacy platform can be retired with confidence.

12. Recommended BeyondX IMS Modernization Lifecycle

- 1 INVENTORY DBDGEN / PSBGEN / PCB / SENSEG
- 2 INVENTORY COBOL / PL/I / JCL / I-O COPYBOOKS / SSAs
- 3 RECONCILE CHANGE CONTROL WITH PRODUCTION EVIDENCE
- 4 PROFILE ACTUAL IMS DATA AND HIERARCHICAL POPULATIONS
- 5 BUILD RECURSIVE SEGMENT + DATA-ELEMENT INVENTORY
- 6 DESIGN RELATIONAL / CLOUD TARGET MODEL
- 7 DEFINE DBD-TO-TABLE AND FIELD-TO-COLUMN AUDIT MATRICES
- 8 TRANSFORM DATA + IMS DML / APPLICATION LOGIC
- 9 TEST DATA + BEHAVIORAL EQUIVALENCE
- 10 RECONCILE 100% OF AGREED SOURCE SCOPE
- 11 CUTOVER WITH FALLBACK / OPERATIONAL CONTROLS
- 12 DECOMMISSION ONLY AFTER VALIDATED ACCEPTANCE

Discovery is not a phase - it is a discipline. Validation is not a milestone - it is continuous. New evidence found during development or testing should feed back into the inventory, mappings, and test controls.

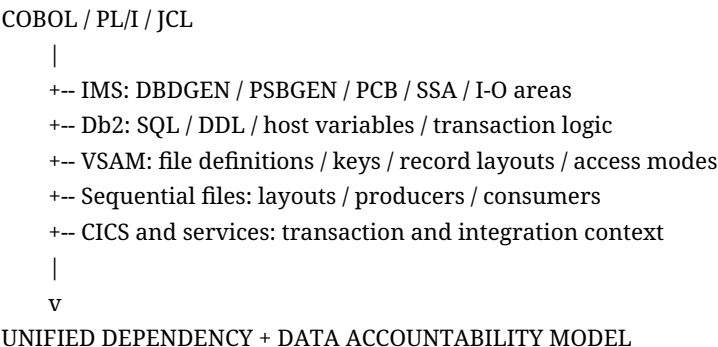
13. Engineering Principles

- A DBD is a source data domain, not necessarily a target table.
- Hierarchy accounting is recursive: every child may have descendants requiring independent target disposition and validation.
- DBDGEN describes what can physically exist; PSBGEN/PCB/SENSEG show application views; SSAs and program logic reveal how data is actually accessed.
- I/O copybooks can be critical surviving documentation, especially where OCCURS and REDEFINES encode record interpretation.
- IMS DML modernization is semantic transformation, not statement-by-statement translation.
- Every row population, data element, and required relationship must be accounted for and auditable.
- 100% accounted does not mean 100% migrated.
- AI can accelerate discovery, correlation, transformation, and testing, but engineering judgment and source-data evidence remain authoritative.
- Modernization succeeds only when the legacy system can be retired with confidence.

14. Mainframe Modernization Extends Beyond IMS

An IMS application rarely exists in isolation. The same COBOL or PL/I estate may use IMS DB/TM, Db2, VSAM, sequential files, CICS transactions, JCL/PROCs, copybooks, and other platform services. Discovery must therefore follow application and data dependencies across technologies rather than stop at the boundary of one DBMS.

A modernization inventory should correlate the code and data architecture together:



The governing rule is that application modernization and data modernization must be analyzed together. COBOL without its IMS, Db2, VSAM, copybook, JCL, transaction, and operational context is only a partial representation of the legacy system.

15. AI and Agentic Tools: Specialized Accelerators, Not Authorities

Modernization teams can use specialized AI and agentic capabilities for code understanding, dependency discovery, business-rule extraction, candidate transformation, SQL and service generation, test generation, and engineering workflows. Representative tool families include IBM mainframe-focused AI/code assistants, AWS mainframe modernization capabilities, Google Cloud mainframe modernization capabilities, and general software-engineering agents such as OpenAI Codex. Product capabilities evolve; tool selection should be validated against the specific source estate and security requirements.

Agent / capability role	Useful evidence and tasks
Mainframe discovery agent	Correlate DBDGEN, PSBGEN, PCB/SENSEGEN, SSAs, DL/I calls, copybooks, JCL, Db2 and VSAM references.
COBOL / PL/I analysis agent	Explain program flow, identify business rules, trace data movement, classify database/file operations and locate error handling.
Data-structure agent	Parse I/O areas, OCCURS, REDEFINES, discriminator fields, record layouts and candidate relational entities.

Transformation agent	Propose SQL, services, modern-language code, target schemas and refactoring patterns from validated requirements.
Testing / reconciliation agent	Generate comparison tests, boundary cases, data-quality checks, row/column reconciliation and behavioral-equivalence tests.
Documentation agent	Generate traceability views and keep mappings, decisions, exceptions and evidence synchronized.

The preferred architecture is multiple specialized agents operating against one authoritative modernization inventory and reconciliation model. No agent should become the system of record for completeness.

15.1 ChatGPT and OpenAI in the Modernization Workbench

ChatGPT can serve as a broad engineering workbench around the specialized mainframe tools. It is not an IMS control-block parser or a substitute for a mainframe engineer; its value is in reasoning across supplied artifacts, explaining unfamiliar code, developing transformation hypotheses, generating and reviewing SQL or modern application code, designing reconciliation logic, creating test cases, analyzing structured data, and producing traceable engineering documentation.

For modernization teams, the OpenAI ecosystem can be viewed as several complementary capabilities: ChatGPT for interactive reasoning and artifact analysis; Codex for repository-scale coding, code review, refactoring and test work; file and data-analysis capabilities for examining structured extracts and reconciliation results; connected apps/plugins for bringing authorized enterprise context into the workflow; Deep Research for evidence-backed external research; and agentic capabilities such as ChatGPT Work or the Agents API for longer-running, tool-using workflows. Availability varies by plan, workspace configuration, permissions and deployment model.

ChatGPT / OpenAI capability	Potential modernization use
ChatGPT	Cross-artifact reasoning over COBOL/PL/I snippets, copybooks, mappings, SQL, test evidence and architecture decisions; explanation, drafting and iterative engineering review.
Codex	Repository-oriented code understanding, implementation, refactoring, code review, test generation and forward engineering under human-controlled access and review.
File & data analysis	Analyze uploaded CSV/XLSX/PDF/DOCX/text artifacts, compute reconciliation results, inspect

	exceptions, generate tables/charts and review code-backed calculations.
Connected apps / plugins	Bring authorized enterprise files, repositories and business context into the workflow subject to existing permissions and workspace controls.
Deep Research	Research current vendor documentation, target-platform behavior, standards and migration options with source-backed reporting.
Work / Agents API	Orchestrate longer-running, multi-step workflows that use tools, files and code while preserving explicit engineering checkpoints and governance.

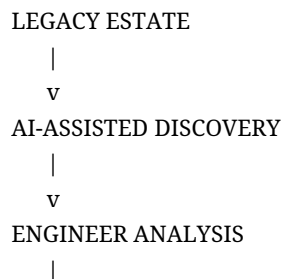
A practical pattern is to provide ChatGPT or a coding agent with a governed evidence package - for example DBDGEN/PSBGEN extracts, COBOL programs, PCB masks, I/O copybooks, SSA definitions, Db2 DDL/SQL, VSAM layouts, JCL and approved source-to-target mappings - and ask it to produce candidate dependency maps, transformation logic and tests. The outputs then enter the same engineering review and reconciliation controls as any other tool-generated artifact.

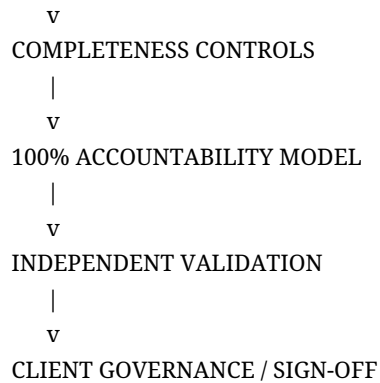
The governing principle remains unchanged: AI output is a hypothesis or accelerator until source evidence and accountable engineers validate it. No ChatGPT, coding agent, vendor transformer or discovery product can establish 100% completeness unless the program first establishes and reconciles the authoritative source inventory.

16. The Accountability Gap in AI-Assisted Modernization

A sophisticated AI tool can produce high-quality analysis and the modernization program can still fail if the engineering team has not established the complete source scope. AI output is evidence for engineering analysis; it is not evidence of completeness.

For example, if an agent successfully analyzes 9,700 programs but the governed inventory contains 10,000 programs, the quality of the 9,700 analyses does not account for the missing 300. The denominator must be established independently through inventory, production evidence, dependency analysis, reconciliation, and exception management.





A modernization milestone should therefore be accepted on evidence, not on the statement that an AI tool completed its analysis. Engineering teams remain accountable for discovery completeness, transformation correctness, exception resolution, and test evidence. Clients remain accountable for requiring objective proof before accepting milestones.

17. Evidence-Based Completeness Controls

The following controls illustrate the level of measurable accountability expected for the agreed modernization scope. The objective is not to force migration of every artifact; it is to establish a documented disposition for every in-scope artifact and data population.

Control domain	Required evidence
IMS metadata	100% of in-scope DBDGEN, segments, PSBGEN, PCB/SENSEGEN and defined hierarchical paths inventoried and dispositioned.
IMS application access	Relevant SSAs, DL/I calls, PCB masks, segment I/O areas and status/error handling correlated to programs and target behavior.
COBOL / PL/I	In-scope programs inventoried, dependency-mapped, classified and reconciled with production/change-control evidence.
Copybooks	I/O and record layouts inventoried; OCCURS, REDEFINES and discriminators analyzed and mapped.
Db2	SQL/DDDL, host variables, tables/views, transaction dependencies and target disposition accounted.
VSAM / files	Datasets, keys, record layouts, access patterns, producers/consumers and target disposition

	accounted.
JCL / operations	Jobs, steps, PROCs, schedules, utilities, dependencies and restart/recovery behavior accounted.
Data	Every in-scope population and data element has a migrated, transformed, consolidated, archived, retained, excluded, or unresolved disposition.
Testing	Data reconciliation and behavioral-equivalence evidence traceable to source requirements and exceptions.

UNKNOWN remains a valid temporary status, but it must remain visible and governed until evidence supports a final disposition.

18. BeyondX Governance Principle

AI proposes. Engineers analyze. Evidence validates. Clients govern.

Accountability cannot be delegated to an AI tool. Tools accelerate modernization, but they do not replace engineering judgment, intellectual ownership, or the obligation to prove completeness. The same standard should apply regardless of integrator size, tool vendor, or delivery model.

This principle complements the core BeyondX position: Discovery is not a phase - it is a discipline. Validation is not a milestone - it is continuous. Modernization is successful only when the legacy system can be retired with confidence.

Publication note. Product capabilities evolve quickly. Tool references in this paper illustrate where specialized and general-purpose AI capabilities may assist modernization; they do not replace project-specific technical evaluation, source evidence, security review, or accountable engineering judgment.

Selected Product References

OpenAI. “Data analysis with ChatGPT.” OpenAI Help Center. Accessed September 2026.

OpenAI. “Working with files in ChatGPT.” OpenAI Academy. Updated 2026.

OpenAI. “ChatGPT is now a partner for your most ambitious work.” July 2026.

OpenAI. “Introducing the Agents API.” September 2026.

OpenAI. “Deep research.” OpenAI Academy. Updated September 2026.

OpenAI. “Connected apps in ChatGPT.” OpenAI Help Center. Updated September 2026.

Conclusion

IMS modernization requires simultaneous understanding of hierarchical data, application views, COBOL data structures, navigational DML, operational evidence, and target relational behavior. In real estates, that analysis must also cross Db2, VSAM, JCL, CICS, files, and other dependencies. AI and agentic tools can materially accelerate this work, but they do not establish completeness or assume accountability.

A disciplined modernization program inventories recursively, correlates across technologies, maps explicitly, transforms semantically, tests comparatively, reconciles quantitatively, and documents every disposition. Engineers remain accountable for correctness and completeness; clients govern acceptance through objective evidence. This is the standard required to retire legacy platforms with confidence.



BeyondX Enterprise Modernization Series

Practical engineering guidance for legacy, data, application, and cloud modernization.

PUBLICATION SERIES

TP-001 Understanding Legacy Data Architectures

TP-002 Flattening Oracle Nested Tables for Cloud Modernization

TP-003 Oracle DATE / UTC Cloud Modernization

TP-004 Mainframe Data Architecture - IMS

THE BEYOND X PROMISE

Discovery is not a phase - it is a discipline.

Validation is not a milestone - it is continuous.

Tools accelerate modernization - but they do not replace engineering judgment.

Modernization is successful only when the legacy system can be retired with confidence.

Technical papers and supporting materials are published through the BeyondX Enterprise Modernization Series.

Technology with Purpose. Opportunity with Impact.

Imagination to Implementation.

© 2026 BeyondX LLC. All rights reserved.